



Lista de Exercícios Dependabilidade e Teste de Software

Prof. Iaçanã Ianiski Weber

Parte 1 — Conceitos de Dependabilidade

Exercício 1. Marque a alternativa que descreve corretamente a cadeia *fault* $\rightarrow$ *error* $\rightarrow$ *failure*:

- (A) Um *fault* é sempre detectável imediatamente; um *error* é o estado incorreto resultante; uma *failure* é o defeito presente no código-fonte.
- (B) Um *fault* é um defeito latente no sistema; um *error* é um estado incorreto ativado pelo *fault*; uma *failure* ocorre quando o *error* compromete o serviço entregue ao usuário.
- (C) Uma *failure* precede o *error*, que por sua vez ativa o *fault* durante a execução.
- (D) *Fault* e *failure* são sinônimos; *error* é o mecanismo de detecção utilizado para identificá-los em tempo de execução.

Gabarito

Alternativa **(B)**. O *fault* é latente; é ativado e gera um *error* (estado interno incorreto); o *error* pode ou não propagar-se até uma *failure* (serviço incorreto observável). As demais alternativas invertem a ordem ou confundem os conceitos.

Exercício 2. Considere o seguinte cenário: um microsserviço de pagamentos processa uma requisição de débito, mas um bug faz com que o saldo seja decrementado duas vezes quando ocorre um timeout seguido de retry pelo cliente.

- a) Identifique o *fault*, o *error* e a *failure* neste cenário.
- b) Proponha um mecanismo para evitar o problema (idempotência, deduplicação ou outro).

Gabarito

- a) *Fault*: bug que não garante idempotência no débito durante retry. *Error*: estado inconsistente do saldo (decrementado duas vezes). *Failure*: cobrança duplicada entregue ao cliente.
- b) Idempotência com *idempotency key*: antes de debitar, verificar se o `pedido_id` já consta na tabela de deduplicação; se sim, retornar resultado anterior sem novo débito.

Parte 2 — Confiabilidade e Disponibilidade

Exercício 3. Um servidor operou continuamente por 60 dias. Nesse período, ocorreram 4 falhas. Os tempos de reparo foram: 2,0h; 1,0h; 3,0h; 0,5h.

- Calcule o MTBF (em horas).
- Calcule o MTTR.
- Calcule a disponibilidade A.
- Estime o *downtime* anual desse sistema.

Gabarito

- $MTTR = 6,5/4 = 1,625$ h. Tempo operacional: $60 \times 24 - 6,5 = 1433,5$ h. $MTTF = 1433,5/4 = 358,375$ h. $MTBF = MTTF + MTTR = \mathbf{360}$ h.
- $MTTR = (2,0 + 1,0 + 3,0 + 0,5)/4 = \mathbf{1,625}$ h.
- $A = MTTF/MTBF = 358,375/360 \approx \mathbf{99,55\%}$.
- Downtime anual $\approx (1 - 0,9955) \times 8760 \approx \mathbf{39,4}$ h/ano.

Exercício 4. Considere dois sistemas reparáveis:

- Sistema P: MTBF = 800 h, MTTR = 16 h
- Sistema Q: MTBF = 200 h, MTTR = 1 h

- Calcule a disponibilidade de cada sistema.
- Qual sistema é mais **confiável**? E qual é mais **disponível**?
- Explique, em termos práticos, por que confiabilidade e disponibilidade podem apontar para sistemas diferentes.

Gabarito

- P: $A = 800/(800 + 16) \approx \mathbf{98,0\%}$; Q: $A = 200/(200 + 1) \approx \mathbf{99,5\%}$.
- Mais **confiável**: P (MTBF = 800 h); mais **disponível**: Q ($A \approx 99,5\%$).
- Um sistema pode falhar com frequência, mas ser reparado rapidamente (alta disponibilidade, baixa confiabilidade). Outro falha raramente, porém demora muito para ser restaurado (alta confiabilidade, disponibilidade moderada).

Exercício 5. Assumindo o modelo exponencial de falhas com MTBF = 1000 h:

- Qual a taxa de falha λ ?
- Calcule $R(200)$, ou seja, a probabilidade de operar sem falha por 200 horas.
- Calcule $R(1000)$.
- Se a missão exige confiabilidade de pelo menos 95% em 100 horas, esse sistema atende?

Gabarito

- $\lambda = 1/1000 = \mathbf{0,001}$ falhas/h.
- $R(200) = e^{-0,2} \approx \mathbf{81,9\%}$.
- $R(1000) = e^{-1} \approx \mathbf{36,8\%}$.
- $R(100) = e^{-0,1} \approx 90,5\% < 95\%$ — **não atende**.

Exercício 6. Uma empresa deseja garantir um SLA de $A \geq 99,9\%$ para sua plataforma. O sistema atual tem $MTBF = 500$ h.

- a) Qual deve ser o MTTR máximo para atender esse SLA?
- b) Com esse MTTR máximo, estime o *downtime* anual permitido.
- c) Se na prática o $MTTR = 2$ h, o SLA é atendido? Justifique.

Gabarito

- a) $500/(500 + MTTR) \geq 0,999 \Rightarrow MTTR \leq 0,5/0,999 \approx 0,5$ h (30 min).
- b) Downtime anual permitido: $0,001 \times 8760 \approx 8,76$ h/ano.
- c) $A = 500/502 \approx 99,60\% < 99,9\%$ — **SLA não atendido.**

Exercício 7. Indique **V** (verdadeiro) ou **F** (falso) para cada afirmação sobre MTTF e MTBF:

- () O MTTF é utilizado em sistemas não reparáveis, enquanto o MTBF é a métrica adequada para sistemas reparáveis.
- () $MTBF = MTTF$, pois ambos medem o tempo médio até a primeira falha do sistema.
- () Para um disco rígido de servidor que pode ser substituído, o MTBF é a métrica mais apropriada.
- () Um MTBF alto implica necessariamente alta disponibilidade, independentemente do valor do MTTR.

Gabarito

- (1) **V** (2) **F** (3) **V** (4) **F**
- (2) São distintos para sistemas reparáveis. (4) $A = MTBF/(MTBF + MTTR)$; um MTTR alto pode manter A baixa mesmo com MTBF elevado (cf. Ex. 10, sistema P com $A \approx 98\%$ apesar de $MTBF = 800$ h).

Exercício 8. Um sistema deve operar continuamente por 48 horas para uma missão crítica. Duas opções estão disponíveis:

- Opção A: $MTBF = 400$ h
- Opção B: $MTBF = 1200$ h

- a) Calcule $R(48)$ para cada opção (modelo exponencial).
- b) Qual o ganho em pontos percentuais de confiabilidade da opção B sobre a opção A?
- c) Se o custo da opção B é $3\times$ maior, você recomendaria a troca? Justifique considerando o contexto de missão crítica.

Gabarito

- a) $R_A(48) = e^{-48/400} = e^{-0,12} \approx 88,7\%$; $R_B(48) = e^{-48/1200} = e^{-0,04} \approx 96,1\%$.
- b) Ganho: $96,1 - 88,7 = 7,4$ pontos percentuais.
- c) **Sim, recomenda-se B:** em missão crítica, 7,4 p.p. adicionais de confiabilidade podem evitar falhas cujo custo humano ou operacional supera em muito o custo $3\times$ maior do componente.

Parte 3 — Safety, Security e Resiliência

Exercício 9. Marque a alternativa que diferencia corretamente *safety* e *security* no contexto de dependabilidade:

- (A) *Safety* refere-se à proteção contra ameaças intencionais de agentes externos; *security* trata de falhas acidentais que podem causar danos ao ambiente.
- (B) *Safety* preocupa-se com a proteção do ambiente contra consequências catastróficas de falhas acidentais do sistema; *security* protege o sistema contra ameaças e ações maliciosas intencionais.
- (C) *Safety* e *security* são sinônimos no contexto de dependabilidade, ambos referindo-se à ausência de consequências catastróficas.
- (D) *Security* é um subconjunto de *safety*, pois toda vulnerabilidade de segurança representa também um risco de *safety*.

Gabarito

Alternativa **(B)**. A diferença central é a origem da ameaça: *safety* trata de falhas *acidentais* (ex.: bug, falha de hardware) que podem causar danos físicos; *security* trata de ameaças *intencionais* (ex.: atacante, malware). As alternativas A e C invertem ou igualam os conceitos; D é incorreta porque a relação não é de subconjunto.

Exercício 10. Considere um sistema de controle de tráfego ferroviário. Um atacante compromete a rede de comunicação e injeta comandos falsos de sinalização.

- a) Esse cenário é primariamente um problema de *safety* ou de *security*? Justifique.
- b) Explique como uma vulnerabilidade de *security* pode se tornar um risco de *safety*.
- c) Proponha pelo menos duas mitigações que enderecem tanto o aspecto de *safety* quanto o de *security*.

Gabarito

- a) Primariamente **security** (ameaça intencional de um atacante), com consequência direta de **safety** (risco de colisão ou descarrilamento).
- b) A vulnerabilidade de *security* permite injeção de comandos falsos; o sistema de controle recebe informações incorretas, o que leva a uma decisão errada de sinalização e, em seguida, ao acidente físico.
- c) Autenticação mútua entre nós de sinalização (certificados digitais) + watchdog de segurança independente que valida coerência dos comandos recebidos em relação ao estado físico dos trilhos.

Exercício 11. Explique o conceito de *hazard* e diferencie-o de *failure*. Em seguida, para um sistema de piloto automático de drone de entrega, identifique:

- a) Dois possíveis *hazards*.
- b) Para cada *hazard*, uma *safety constraint* (restrição de segurança funcional).
- c) Uma mitigação arquitetural para cada *hazard*.

Gabarito

Hazard: condição do sistema ou ambiente que pode levar a um acidente (dano físico). Diferente de *failure*: um *hazard* pode existir sem causar acidente se as condições não se combinarem; uma *failure* é o serviço incorreto entregue.

- a) (i) Perda de sinal GPS em área urbana; (ii) Falha no sensor de altitude.

- b) SC(i): o drone não deve voar acima da altitude máxima sem confirmação de altitude válida; SC(ii): em caso de leitura de altitude inconsistente, iniciar procedimento de pouso de emergência imediatamente.
- c) Mit.(i): fusão de GPS + inercial (IMU) com detecção automática de falha de GPS; Mit.(ii): dois sensores de altitude independentes com votação majoritária (2-of-3 voting).

Exercício 12. Indique **V** (verdadeiro) ou **F** (falso) para cada afirmação sobre as propriedades CIA e AAA:

- () Confidencialidade (CIA) é violada quando dados de pacientes são acessados por pessoal não autorizado de outro setor hospitalar.
- () Integridade (CIA) refere-se à disponibilidade contínua do sistema de prontuário, garantindo acesso ininterrupto 24 horas por dia.
- () Disponibilidade (CIA) é violada quando um ataque de negação de serviço (DoS) torna o sistema de prontuário inacessível.
- () Autenticação (AAA) determina quais recursos e operações um usuário já identificado pode executar no sistema.
- () Accountability (AAA) permite rastrear quem acessou ou modificou um prontuário eletrônico e em que momento.

Gabarito

(1) **V** (2) **F** (3) **V** (4) **F** (5) **V**

(2) **Integridade** garante que os dados não foram alterados indevidamente; disponibilidade contínua é a propriedade *Availability*. (4) **Autenticação** verifica *quem* é o usuário; *Autorização* (não autenticação) define o que o usuário autenticado pode acessar.

Exercício 13. Explique por que projetar safety e security separadamente pode ser arriscado. Dê um exemplo concreto em que uma mitigação de security cria um novo risco de safety (ou vice-versa).

Gabarito

Projetar separadamente ignora interações entre as duas propriedades: uma solução que melhora security pode criar gargalos ou pontos de falha que comprometem safety, e vice-versa.

Exemplo: um painel de controle industrial exige autenticação multifator para todos os comandos (security). Em uma emergência real, o delay ou falha da autenticação impede o acionamento imediato do desligamento de segurança (safety), podendo agravar o acidente. Solução: projetar um *safety override* autenticado por canal físico independente.

Parte 4 — Teste de Software: Caixa Preta

Exercício 14. Uma função recebe um inteiro *idade* e retorna uma string conforme as regras:

- Se $0 \leq \text{idade} \leq 12$: retorna “criança”
- Se $13 \leq \text{idade} \leq 17$: retorna “adolescente”
- Se $18 \leq \text{idade} \leq 64$: retorna “adulto”
- Se $65 \leq \text{idade} \leq 150$: retorna “idoso”

- a) Identifique todas as classes de equivalência (válidas e inválidas).
- b) Projete um caso de teste representativo para cada classe.
- c) Identifique todas as fronteiras e projete casos de teste para os valores *on* e *off* de cada fronteira.

Gabarito

- a) Classes válidas: $[0,12]$, $[13,17]$, $[18,64]$, $[65,150]$. Classes inválidas: $(-\infty, -1]$, $[151, +\infty)$.
- b) CTs representativos: 6 retorna “criança”; 15 retorna “adolescente”; 30 retorna “adulto”; 100 retorna “idoso”; -5 retorna “inválido”; 200 retorna “inválido”.
- c) Fronteiras e pares *on/off*: $(-1/0)$; $(12/13)$; $(17/18)$; $(64/65)$; $(150/151)$. Total: 10 casos de teste de fronteira.

Exercício 15. Considere a seguinte especificação de uma função que calcula o desconto em uma loja online:

```
float calcular_desconto(float valor, int tipo_cliente, int cupom);
```

Parâmetro	Tipo	Domínio	Descrição
valor	float	$(0, 10000]$	Valor da compra em reais
tipo_cliente	int	$\{1, 2, 3\}$	1=normal, 2=premium, 3=VIP
cupom	int	$\{0, 1\}$	0=sem cupom, 1=com cupom

Regras:

- Se qualquer parâmetro estiver fora do domínio, retornar -1.
- Desconto base: normal = 0%, premium = 10%, VIP = 20%.
- Se *cupom* = 1, adicionar 5% ao desconto.
- Desconto máximo: 25%. Se o desconto calculado ultrapassar 25%, limitar a 25%.
- Retorno: valor do desconto em reais ($\text{valor} \times \text{percentual}$).

- a) Identifique todas as classes de equivalência inválidas.
- b) Identifique todas as classes de equivalência válidas (considere as combinações de tipo de cliente e cupom).
- c) Identifique as fronteiras relevantes e projete casos de teste para valores *on* e *off*.
- d) Monte uma suíte de testes consolidada em tabela com colunas: ID, valor, tipo_cliente, cupom, resultado esperado, técnica, classe coberta.

Gabarito

- a) Inválidas: $\text{valor} \leq 0$; $\text{valor} > 10000$; $\text{tipo_cliente} \notin \{1, 2, 3\}$; $\text{cupom} \notin \{0, 1\}$.
- b) Válidas: (normal, s/cupom): 0%; (normal, c/cupom): 5%; (premium, s/cupom): 10%; (premium, c/cupom): 15%; (VIP, s/cupom): 20%; (VIP, c/cupom): 25% (cap aplicado).
- c) Fronteiras: $\text{valor} = 0$ (off), 0,01 (on), 10000 (on), 10000,01 (off). **cupom**: -1 (off), 0 (on), 1 (on), 2 (off).
- d) Suíte (exemplos principais):

ID	valor	tipo	cupom	esperado	técnica	classe
T01	100	1	0	0,00	CE válida	normal, s/cupom
T02	100	1	1	5,00	CE válida	normal, c/cupom
T03	100	2	0	10,00	CE válida	premium, s/cupom
T04	100	2	1	15,00	CE válida	premium, c/cupom
T05	100	3	0	20,00	CE válida	VIP, s/cupom
T06	100	3	1	25,00	CE válida	VIP, c/cupom (cap)
T07	0	1	0	-1	Fronteira off	valor inválido
T08	0,01	1	0	0,00	Fronteira on	valor mínimo
T09	10000	1	0	0,00	Fronteira on	valor máximo
T10	10001	1	0	-1	Fronteira off	valor inválido
T11	100	0	0	-1	CE inválida	tipo inválido
T12	100	1	2	-1	CE inválida	cupom inválido

Exercício 16. Para a especificação do Exercício 15, identifique pelo menos 3 defeitos plausíveis que um programador poderia introduzir na implementação. Para cada defeito, indique qual caso de teste da sua suíte o detectaria.

Gabarito

- (1) Usar $>$ em vez de $\geq$ na verificação do domínio de **valor** ($\text{valor} > 0$): detectado por T08 ($\text{valor} = 0,01$ deve retornar 0, não -1).
- (2) Omitir o cap de 25%: detectado por T06 (VIP+cupom deveria retornar 25,00 e não 26,00).
- (3) Inverter percentuais por tipo (ex.: normal recebe 10%): detectado por T01 (deve retornar 0,00) e T03 (deve retornar 10,00).

Exercício 17. Uma função recebe três inteiros representando os lados de um triângulo e classifica-o como equilátero, isósceles, escaleno ou erro (não forma triângulo). Sabe-se que a implementação contém o seguinte defeito: a condição de desigualdade triangular usa $\geq$ em vez de $>$ (ou seja, $a+b \geq c$ em vez de $a+b > c$).

- Qual caso de teste detecta esse defeito? Forneça valores concretos.
- Explique por que um caso de teste com valores “genéricos” (ex.: $a = 3, b = 4, c = 5$) **não** detectaria esse defeito.
- Qual técnica de teste (particionamento ou valor limite) é mais eficaz para encontrar esse tipo de bug?

Gabarito

- CT: $a = 3, b = 4, c = 7$. Com $\geq$: $3 + 4 \geq 7$ (verdadeiro — aceita como triângulo, incorreto). Com $>$: $3 + 4 \not\geq 7$ (falso — retorna erro, correto). O CT detecta o defeito.
- $a = 3, b = 4, c = 5$: $3 + 4 = 7 > 5$ — ambas as condições ($>$ e $\geq$) produzem o mesmo resultado (aceita), portanto o bug não é exercido.
- Valor limite** (*boundary value analysis*): o defeito reside exatamente na fronteira da desigualdade triangular ($a + b = c$).

Parte 5 — Teste de Software: Caixa Branca

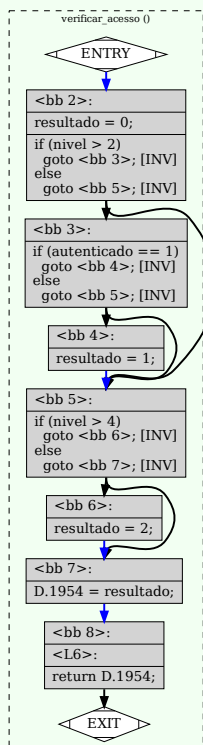
Exercício 18. Considere o seguinte código em C:

```
1  int verificar_acesso(int nivel, int autenticado) {  
2      int resultado = 0;  
3      if (nivel >= 3 && autenticado == 1) {  
4          resultado = 1;  
5      }  
6      if (nivel >= 5) {  
7          resultado = 2;  
8      }  
9      return resultado;  
10 }
```

a) Desenhe o grafo de fluxo de controle (CFG) deste código.

Gabarito

a) CFG gerado com `gcc -fdump-tree-cfg-graph`:



b) CT1: nivel=2, aut=0 retorna 0; CT2: nivel=3, aut=1 retorna 1; CT3: nivel=5, aut=1 retorna 2.

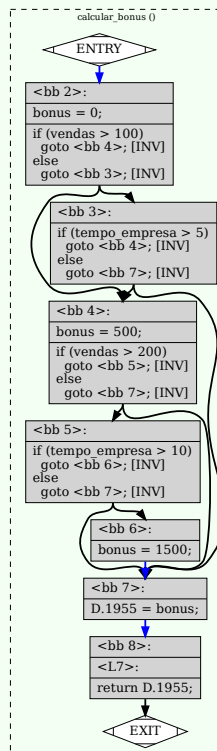
Exercício 19. Considere o seguinte código:

```
1 int calcular_bonus(int vendas, int tempo_empresa) {  
2     int bonus = 0;  
3     if (vendas > 100 || tempo_empresa > 5) {  
4         bonus = 500;  
5         if (vendas > 200 && tempo_empresa > 10) {  
6             bonus = 1500;  
7         }  
8     }  
9     return bonus;  
10 }
```

- a) Desenhe o grafo de fluxo de controle (CFG).
- b) É possível alcançar 100% de cobertura de caminhos com menos de 4 casos de teste? Justifique.

Gabarito

- a) CFG gerado com gcc -fdump-tree-cfg-graph:



- b) **Não**: existem exatamente 3 caminhos distintos, portanto são necessários pelo menos 3 CTs para path coverage também — o mesmo número que branch coverage neste caso.

Exercício 20. Considere o seguinte código:

```

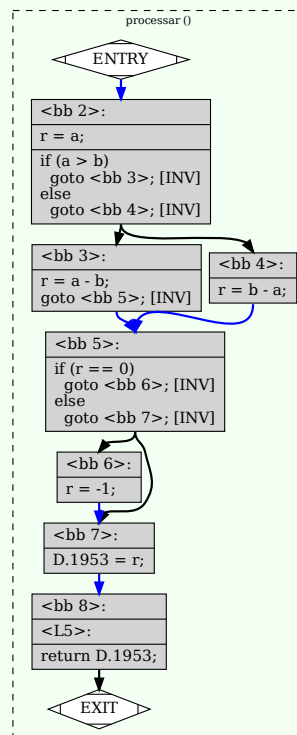
1 int processar(int a, int b) {
2     int r = a;
3     if (a > b) {
4         r = a - b;
5     } else {
6         r = b - a;
7     }
8     if (r == 0) {
9         r = -1;
10    }
11    return r;
12 }

```

- Identifique todos os caminhos possíveis no código.
- Projete casos de teste para cobertura de caminhos (path coverage) de 100%.
- Existe algum caminho infactível? Se sim, qual e por quê?

Gabarito

CFG gerado com `gcc -fdump-tree-cfg-graph`:



- P1: $a > b, r \neq 0$; P2: $a > b, r = 0$ (infactível); P3: $a \leq b, r \neq 0$; P4: $a \leq b, r = 0$ ($a = b$).
- CT1: $a = 5, b = 3$ retorna 2 (P1); CT2: $a = 3, b = 3$ retorna -1 (P4); CT3: $a = 2, b = 5$ retorna 3 (P3). (P2 é infactível — ver c)
- P2 é infactível:** $a > b$ (estrito) implica $a \neq b$, logo $r = a - b \neq 0$ — contradição com $r = 0$.